

Module 6 Security Performance and Empirical Evidence

Overview and study route

Software Engineering in the Era of AI | Master's year one | Electrical and Information Engineering | Weeks 13 and 14 | 2026 to 2027

An AI-enabled service can be fast, fluent and wrong. It can also be numerically correct while exposing data or permitting an unauthorised action. This module treats security, performance and evaluation as related engineering responsibilities with separate evidence requirements.

Week 13 develops a threat model and governance decisions for the simulated advice service. Week 14 measures latency, cost and quality and interprets uncertainty. The master's-level task is to defend a decision under incomplete evidence, stating what was measured, what remains uncertain and which controls address the relevant failures.

Learning outcomes

- Identify assets, actors, trust boundaries and plausible failure paths.
- Explain prompt injection, excessive agency and improper output handling in a bounded application.
- Map a risk to an enforced control and a verification case.
- Distinguish governance guidance from legal or certification claims.
- Measure latency and cost with explicit definitions and denominators.
- Apply profiling, Amdahl's law and simple concurrency reasoning.
- Interpret classification results, sample uncertainty and comparative experiments.

The module supports LO3, LO4, LO5, LO6 and LO7.

1 Threat modelling

A threat model describes what needs protection, who or what could affect it, how information and authority move, and what could go wrong. Start with assets: telemetry integrity, approved manuals, credentials, evaluation records, compute budget and users' confidence in the report. Identify actors such as a legitimate student, an unauthorised user, a compromised dependency and an external content source.

Draw trust boundaries around the user input, document store, model provider and tool executor. A text passage crossing from a manual into model context should not gain the authority of an application instruction. A model response crossing into a database query or report should be validated for its destination.

Threats are scenarios, not only labels. Example: a malicious passage in an imported manual instructs the model to request a file outside the authorised document set. The asset is restricted information; the path is retrieved text influencing a tool request; the control is an enforced allowlist and authorisation check at the tool boundary. A prompt warning can supplement this control but cannot replace it.

2 Current AI security concerns

The published OWASP 2026 LLM Top 10 identifies prompt injection, sensitive information disclosure, excessive agency, supply chain risks, poisoning, unbounded consumption, misinformation, hidden context exposure, vector and embedding weaknesses, and improper output handling. [R1] Use these categories to prompt analysis, not as a complete threat model or a guarantee that every application faces equal risk.

For the course service, several paths deserve concrete attention: untrusted manual content influencing the model; an advice response being treated as a command; an unknown dependency entering a build; unrestricted retry loops spending resources; and old or unauthorised passages appearing in retrieval results. The controls must be placed where ordinary application code can enforce them.

3 Prompt injection and data boundaries

Prompt injection occurs when attacker-controlled content attempts to redirect a model's behaviour. Direct injection is supplied as an interaction request; indirect injection can be embedded in retrieved documents, web pages, code comments or tool results. The model sees text from different sources, and natural-language boundaries alone are imperfect security boundaries.

Use layered controls. Keep tool privileges narrow. Validate arguments and authorised resources outside the model. Separate secret storage from model-visible context. Limit network destinations and filesystem access where appropriate. Review consequential actions before execution according to the application policy. Test realistic failure cases in a disposable environment.

The classroom probe is harmless text inside an invented manual asking for an unauthorised document. The expected result is that the tool boundary denies the request. This demonstrates an enforcement property even if the model repeats the instruction. Do not grade only whether the model's wording appears obedient; inspect the actual permitted effects.

Filtering one phrase such as "ignore previous instructions" cannot establish resistance to all injections. Attack text can be phrased differently, encoded or distributed across context. The defence claim should be limited to the tested scenarios and the enforced privileges.

4 Output handling and excessive agency

Treat model output as untrusted input to the next component. Do not pass an explanation directly to eval, a shell command or an SQL string. Use a fixed set of named operations with typed, validated arguments. The application selects whether the operation is permitted for the current user and task.

Excessive agency means that the system can take actions beyond what its purpose requires. An advice reader needs access to approved passages; it does not need permission to alter equipment settings or delete files. Reducing available actions narrows the consequence of a mistaken or manipulated request.

The numerical path in this course remains deterministic. Model-generated advice is explanatory and must not override thresholds, units or validated calculations. A system intended for real equipment control would require a separate safety engineering process and evidence beyond this classroom prototype.

Control matrix

Scenario	Enforced control	Verification evidence
Request for unapproved document	Authorisation check on document ID	Denied call and unchanged data
Model returns executable text	Treat response as data; no evaluation of code	Text is displayed or rejected, never executed
Repeated provider failure	Retry limit and total deadline	Bounded calls and fallback result
Unsupported source citation	Check returned IDs against retrieved authorised set	Invalid response rejected
Dependency name suggested by AI	Verify official identity and necessity	Reviewed dependency record

These controls address specific paths. A source-ID check, for example, does not prove that the cited passage supports the statement.

5 Privacy supply chain and governance

Data minimisation means collecting and exposing only the data required for the task. A maintenance explanation may need an anonymised fault code and approved manual passage, not a student's identity or a complete device log. Consider retention, access and deletion for prompts, traces and evaluation records.

Supply chain review covers code, packages, model services, datasets and documents. Verify the identity and source of a component before including it. Check licences and institutional terms for intended use rather than assuming generated material is unrestricted. Record these decisions and consult the responsible institutional authority for unresolved legal or contractual questions.

NIST's AI Risk Management Framework provides Govern, Map, Measure and Manage as organising functions; its Generative AI Profile addresses additional generative-AI concerns. [R2] In this course, use them to allocate responsibility, define context, collect evidence and decide controls. They are guidance references, not a declaration that the project is legally compliant or certified.

Ethical evaluation includes who benefits, who bears errors and whether affected users understand the system's limitations. For a maintenance assistant, test ambiguous queries and unfamiliar device models rather than only the easiest known examples. A system should express unsupported conclusions as unavailable, not conceal uncertainty behind fluent language.

6 Performance begins with measurement

Define the workload, environment, configuration and metric before optimising. Record input size, hardware, interpreter, cache state, concurrency, provider configuration and repetition count. Use a monotonic high-resolution clock for elapsed durations. Separate setup cost from the operation being measured unless setup is part of the user-facing task.

A benchmark should measure accepted useful work. If an optimisation skips validation or returns an empty answer, its shorter time is not a fair comparison. Run correctness checks before and after the change and retain the same workload.

The supplied `benchmark_demo.py` repeats a deterministic calculation and reports a median and nearest-rank p95. Its times depend on the local machine. It makes no claim about network latency or model performance. Do not copy illustrative timings into a project as if they were observed.

7 Latency throughput and concurrency

Latency is the duration of one operation from the defined start to finish. Throughput is completed operations per unit time under a stated workload. Concurrency is the number of operations in progress. Increasing concurrency can improve throughput until a bottleneck saturates, after which queueing may worsen latency.

The median describes the middle of a latency distribution. A p95 estimate describes a point at or below which roughly 95% of observations fall under a specified quantile convention. Tail latency matters because a minority of very slow requests can dominate the user experience.

For the nearest-rank convention, sort n values and select the value at position $\text{ceil}(0.95n)$, counting from one. With twenty observations, p95 is the nineteenth sorted value. With only a few observations, tail estimates are unstable; a single run cannot establish a p95.

In a stable system, Little's law relates average items in the system L , average arrival rate λ and average time W as $L = \lambda \times W$. At four requests per second with mean time 0.5 seconds, average in-flight work is two requests. This does not predict the tail or justify operating at a capacity limit; its assumptions and consistent boundaries matter.

8 Profile before optimising

Profiling identifies where time or resources are spent. If model calls dominate end-to-end latency, optimising a tiny list loop may have negligible user impact. If retrieval loads the same file repeatedly, caching a parsed authorised document may be more useful than changing the language model.

Amdahl's law estimates the maximum speedup when only part of a task improves. If fraction f of the original time is accelerated by factor s , total speedup is $1 / ((1 - f) + f/s)$, assuming other costs stay constant. If 80% is improved fourfold, speedup is $1 / (0.2 + 0.8/4) = 2.5$, not four.

Caching trades computation for stored results and invalidation work. Include document revision and permissions in keys where they affect the answer. Batching may improve throughput while increasing waiting time. Asynchronous execution can overlap independent waits but adds cancellation, ordering and failure complexity.

Quantisation and edge inference are extensions. They can alter resource use and accuracy and must be evaluated on the relevant workload. A smaller model may meet a narrow task requirement more economically, but size alone does not establish suitability.

9 Cost per accepted result

Model cost depends on the selected provider's current pricing and usage categories. Use actual reported input and output usage and the applicable dated rates. Include retries, failed calls and evaluation runs. Do not present a hard-coded classroom rate as a live quote.

For an illustrative pricing model, $\text{cost} = \text{input_tokens} / 1,000,000 \times \text{input_rate} + \text{output_tokens} / 1,000,000 \times \text{output_rate}$. If a call uses 2000 input tokens at 2 currency units per million and 500 output tokens at 8 per million, the cost is $0.004 + 0.004 = 0.008$ currency units. These rates are invented for arithmetic practice.

Cost per accepted result divides total relevant expenditure by the number of results meeting the quality threshold. If 100 attempts cost 0.8 units and 80 are accepted, cost per accepted result is 0.01 units. The rejected attempts remain in the numerator.

Compare quality, latency and cost together. A Pareto-dominated option is worse or equal on all considered objectives and strictly worse on at least one, under the same measurement conditions. Where options trade advantages, choose according to the application's constraints rather than declaring one universally best.

10 Classification evidence and unequal consequences

A threshold alarm can be evaluated against independently labelled cases. True positives are actual events correctly flagged; false positives are non-events incorrectly flagged; false negatives are missed events; true negatives are non-events correctly left unflagged.

Precision = $TP / (TP + FP)$ asks how many flagged cases are correct. Recall = $TP / (TP + FN)$ asks how many actual events are detected. Accuracy = $(TP + TN) / \text{total}$ can hide poor event detection when events are rare. Define how zero denominators are reported rather than inventing a number.

Suppose $TP = 8$, $FP = 2$, $FN = 4$ and $TN = 86$. Precision is $8/10 = 0.8$, recall is $8/12$, approximately 0.667, and accuracy is $94/100 = 0.94$. High accuracy does not erase the four missed events. Select thresholds using the consequences of each error and development evidence, then assess on held-out cases.

For generated explanations, use a rubric with separate dimensions such as factual support, task coverage, unit preservation and appropriate abstention. Some failures should be hard rejection conditions. An average score must not hide an unsupported safety claim in otherwise readable prose.

11 Uncertainty and experimental design

Every measured rate has a denominator and a sampling process. Twenty correct outputs out of twenty do not prove zero future error. Under independent identically distributed Bernoulli trials, an exact one-sided 95% upper bound for the failure probability after zero failures in n trials is $1 - 0.05^{(1/n)}$. With $n = 20$, the bound is about 0.139, or 13.9%. The assumptions rarely hold perfectly for related prompts, so explain the limitation.

This bound is not the probability that the system is safe and is not a substitute for domain-specific assurance. It illustrates why small perfect test sets support modest conclusions. Include difficult and no-answer cases rather than sampling only familiar successes.

In a comparison, freeze the test set, label policy and configuration. Repeat stochastic outputs where variation matters. Report all planned trials, including failures. Paired designs can reduce task variability, but order effects and learning need control. A confidence interval quantifies uncertainty under a model; it does not fix biased sampling or leaked evaluation data.

Use ablations to investigate mechanisms. For example, compare keyword retrieval alone, retrieval plus generation and the same generation without retrieval. Keep the evaluation questions and quality threshold fixed. If the no-retrieval system performs equally well, investigate whether the tasks require the documents or whether information leaked into the setup.

Weekly laboratories

Week 13 lab Threat model and controls

- Minutes 0 to 15: identify assets and draw boundaries around documents, model and tools.
- Minutes 15 to 30: select three concrete threat scenarios and their enforcement points.
- Minutes 30 to 45: run harmless local probes for an unapproved source ID and malformed output.
- Minutes 45 to 55: record actual denied actions or fallback behaviour and one remaining limitation.
- Minutes 55 to 60: submit the control matrix and a governance responsibility note.

Use synthetic data and local tools. Do not test attacks against systems or accounts outside the assigned environment.

Week 14 lab Quality performance and cost

- Minutes 0 to 10: state workload, correctness threshold and timing method.
- Minutes 10 to 25: run the supplied benchmark and inspect all observations.
- Minutes 25 to 40: calculate median, p95 and an illustrative cost per accepted result.
- Minutes 40 to 50: compare a proposed optimisation while keeping required checks intact.
- Minutes 50 to 60: report the decision, uncertainty and one follow-up measurement.

Extension: compare two real provider configurations on the frozen project evaluation set, with an explicit spending budget and actual usage records.

Practice questions

Question 1 Trust boundary

A retrieved manual asks the model to read a private file. Where should the decisive control operate?

Model answer. At the file or tool access boundary in ordinary application code, using explicit authorisation and an allowed resource set. The manual is data. A prompt instruction alone cannot enforce filesystem permissions.

Question 2 Schema limitation

Why can a structurally valid response still be dangerous to use as a command?

Model answer. Valid syntax says nothing about authorisation or the safety and meaning of the requested action. Keep output as data unless it maps to an explicitly allowed operation with validated arguments and an appropriate permission decision.

Question 3 Governance

Does citing NIST's AI RMF establish that a project meets all applicable legal requirements?

Model answer. No. It is a risk-management reference. Legal, contractual and institutional requirements depend on context and need separate determination. The project should state which practices were applied and what evidence supports them.

Question 4 Tail latency

For twenty sorted observations, which value is the nearest-rank p95? Why should the report name the convention?

Model answer. It is position $\text{ceil}(0.95 \times 20) = 19$, counting from one. Different quantile conventions can interpolate differently, especially in small samples. Naming the method makes the number reproducible.

Question 5 Amdahl calculation

Half of a task's time is improved by a factor of five. Calculate the total speedup.

Model answer. Speedup is $1 / (0.5 + 0.5/5) = 1/0.6$, approximately 1.667. The unchanged half limits the gain, assuming other costs remain fixed.

Question 6 Concurrency

A stable service receives six requests per second and has mean time 0.4 seconds within the same boundary. Estimate average in-flight requests.

Model answer. Little's law gives $L = 6 \times 0.4 = 2.4$ requests on average. This is not a required integer at every moment and does not describe p95 latency or maximum capacity.

Question 7 Cost arithmetic

Using invented rates of 3 units per million input tokens and 6 per million output tokens, calculate the cost of 1000 input and 500 output tokens.

Model answer. Input cost is 0.003 and output cost is 0.003, totalling 0.006 units. These are illustrative rates, not current provider prices.

Question 8 Accepted-result cost

Sixty attempts cost 1.2 units and forty meet the quality threshold. Find cost per accepted result.

Model answer. $1.2 / 40 = 0.03$ units. Failed attempts remain part of total cost. Also report acceptance rate $40/60$, approximately 66.7%, and the threshold used.

Question 9 Classification

With $TP = 9$, $FP = 3$ and $FN = 1$, calculate precision and recall.

Model answer. Precision is $9/12 = 0.75$. Recall is $9/10 = 0.9$. Accuracy cannot be calculated without TN or the total number of cases.

Question 10 Zero observed failures

Why does zero failure in twenty model trials not justify a claim of perfect reliability?

Model answer. The sample is small and may not represent future cases. Even under independent identical Bernoulli assumptions, the one-sided 95% upper failure bound is about 13.9%. Correlated tasks or selection bias further limit generalisation.

Question 11 Caching

Why should document revision and user permissions matter to an advice cache?

Model answer. The same query can require a different answer after a document update or for a user with different access. Ignoring those factors can return stale or unauthorised content. Include relevant factors in the key and define invalidation.

Question 12 Fair optimisation

A faster version skips input validation and accepts more malformed data. Can its latency be compared as an equivalent improvement?

Model answer. No. It changes the required behaviour and quality threshold. Restore equivalent validation or explicitly analyse the changed contract; speed alone does not establish a better implementation.

Revision checklist

Map a threat to a control and an observed effect. Explain what remains unprotected. Calculate latency, cost and classification measures with denominators. Apply Amdahl's law with assumptions. Distinguish a confidence bound from a safety guarantee. Defend a quality-cost choice using frozen evaluation evidence.

Sources and further reading

- [R1] OWASP GenAI Security Project. Published 2026 LLM Top 10 source repository.
<https://github.com/GenAI-Security-Project/GenAI-LLM-Top10>
- [R2] NIST. AI Risk Management Framework and Generative AI Profile.
<https://www.nist.gov/itl/ai-risk-management-framework>
- Python Software Foundation. time and statistics documentation.
<https://docs.python.org/3/library/time.html> and <https://docs.python.org/3/library/statistics.html>
- Further reading: NIST Engineering Statistics Handbook, binomial confidence bounds.
<https://www.itl.nist.gov/div898/handbook/prc/section2/prc241.htm>
- Further reading: Google Site Reliability Engineering, monitoring distributed systems.
<https://sre.google/sre-book/monitoring-distributed-systems/>

References checked 15 September 2026. Rates, counts and workloads are invented examples unless a student supplies actual measured results. The course exercises do not certify a system for physical equipment operation.